SIMULAÇÃO DE MALWARE: RANSOMWARE & KEYLOGGER COM PYTHON

Descrição do Projeto

O objetivo é demonstrar na prática o funcionamento de duas ameaças digitais comuns, ajudando a compreender os mecanismos de ataque para fortalecer as estratégias de defesa.

1. Ransomware Simulado

- O script utiliza a biblioteca cryptography para realizar a criptografia simétrica de arquivos.
- Funcionamento: O encrypter.py gera uma chave única, criptografa o arquivo alvo e remove o original. O decrypter.py utiliza a mesma chave para restaurar o acesso aos dados.
- Arquivos inclusos: encrypter.py, decrypter.py, chave.key.

2. Keylogger Simulado

- Implementado com a biblioteca pynput para capturar eventos do teclado.
- Funcionamento: O script monitora todas as teclas pressionadas e as registra em um arquivo de log local (log.txt).
- Arquivos inclusos: keylogger.py, log.txt.

Medidas de Prevenção e Defesa

O conhecimento técnico desses ataques nos permite implementar melhores defesas:

Contra Ransomware:

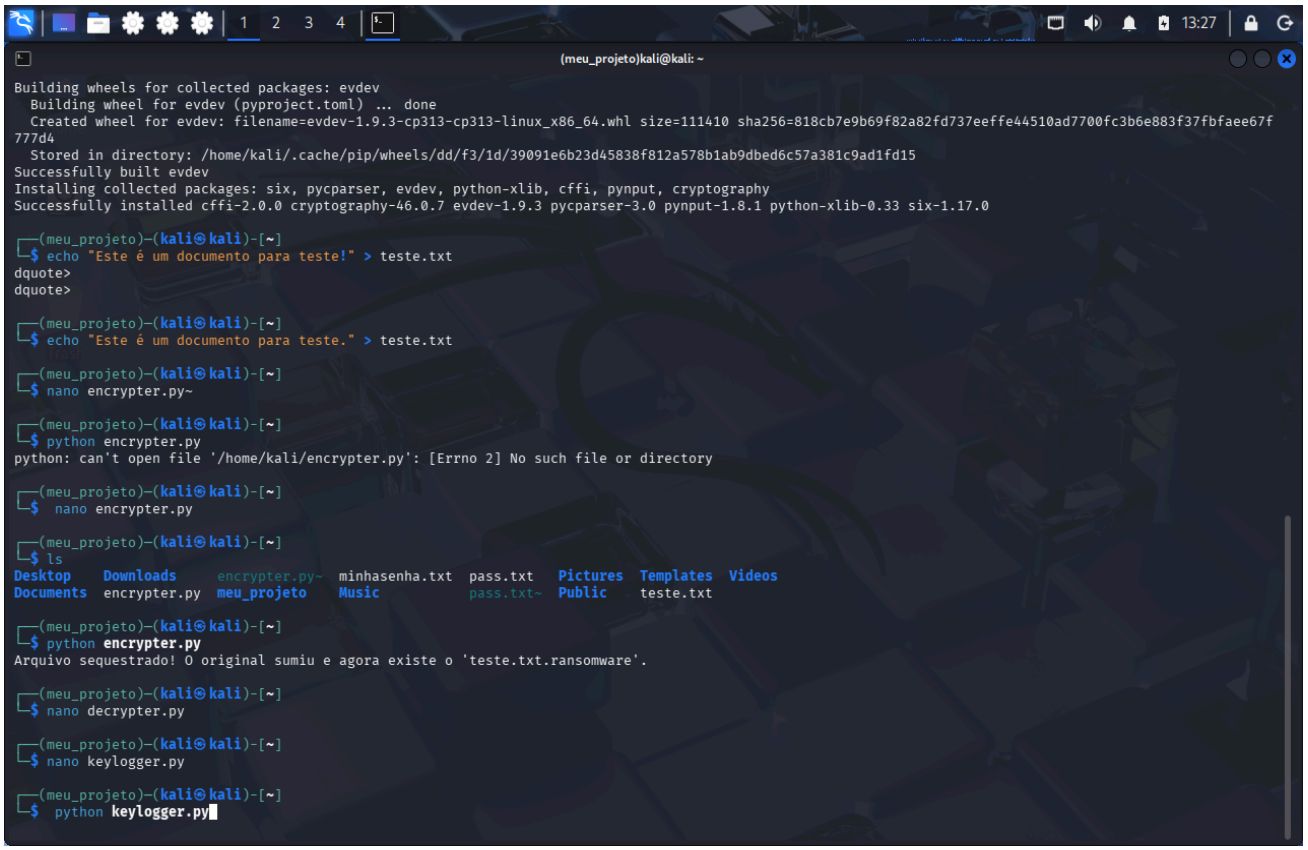
- Backups Offline: Manter cópias de segurança em dispositivos não conectados à rede (o ransomware não pode criptografar o que não alcança).

- Soluções de Endpoint (EDR): Softwares que detectam comportamentos anômalos, como a criptografia rápida de centenas de arquivos.
- Princípio do Menor Privilégio: Usuários não devem ter permissão de escrita em pastas críticas do sistema.

Contra Keyloggers:

- Antivírus Atualizado: A maioria dos keyloggers comuns é detectada por assinaturas de antivírus conhecidas.
- Teclados Virtuais e MFA: O uso de autenticação de dois fatores (2FA) torna a senha capturada inútil para o atacante sem o segundo código.
- Sandboxing: Executar arquivos suspeitos em ambientes isolados para observar seu comportamento antes de abri-los no sistema principal.

Resultado:



```
┌──(meu_projeto)─(kali㉿kali)-[~]
└─$ nano keylogger.py

┌──(meu_projeto)─(kali㉿kali)-[~]
└─$  python keylogger.py

^CTraceback (most recent call last):
  File "/home/kali/keylogger.py", line 8, in <module>
    listener.join()
    ~~~~~~~~~~~~^^
  File "/home/kali/meu_projeto/lib/python3.13/site-packages/pynput/_util/__init__.py", line 295, in join
    super(AbstractListener, self).join(timeout, *args)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/threading.py", line 1094, in join
    self._handle.join(timeout)
    ~~~~~~~~~~~~~~~~~~^^^^^^^^^
KeyboardInterrupt


┌──(meu_projeto)─(kali㉿kali)-[~]
└─$ python keylogger.py
Ciberseguranca
Suelen
^CTraceback (most recent call last):
  File "/home/kali/keylogger.py", line 8, in <module>
    listener.join()
    ~~~~~~~~~~~~^^
  File "/home/kali/meu_projeto/lib/python3.13/site-packages/pynput/_util/__init__.py", line 295, in join
    super(AbstractListener, self).join(timeout, *args)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^
  File "/usr/lib/python3.13/threading.py", line 1094, in join
    self._handle.join(timeout)
    ~~~~~~~~~~~~~~~~~~^^^^^^^^^
KeyboardInterrupt


┌──(meu_projeto)─(kali㉿kali)-[~]
└─$ cat log.txt
Key.enter Key.backspace Key.backspace Key.backspace Key.backspace Key.backspace Key.backspace Key.ctrl 'c' 's' 'u' 'e' 'l' Key.backspace Key.backspace Key.
backspace Key.backspace Key.backspace Key.caps_lock 'c' Key.caps_lock 'i' 'b' 'e' 'r' 's' 'e' 'g' 'u' 'r' 'a' 'n' 'c' 'a' Key.space Key.enter 's' 'u' Key.b
ackspace Key.backspace Key.caps_lock 's' Key.caps_lock 'u' 'e' 'l' 'e' 'n' Key.enter Key.ctrl 'c'

┌──(meu_projeto)─(kali㉿kali)-[~]
└─$
```